

CSIS 2300 Introduction to Algorithms

Spring 2026 – Homework 8 Solutions

Instructions: You do not need to typeset your solutions; however, your submission must be a single PDF file. The assignment is due by **8:59pm on Tuesday, March 31**. Remember that you may submit as many times as you wish before the deadline (we'll only grade the last one), but late submissions will receive no credit – so submit early, submit often!

(1) A fast food chain is considering opening some new restaurants along a long highway. They have a series of n potential locations located at mile markers $m_1 < m_2 < \dots < m_n$. They also have a series of corresponding profit projections $p_1, p_2, \dots, p_n$, one per location; all of the p_i are positive. The chain also does not want restaurants closer than k miles together. (e.g. If $m_7 = 50, m_8 = 65$, and $k = 20$, then we cannot put restaurants at both m_7 and m_8 .)

Your task is to create a dynamic programming algorithm that maximizes the chain's project profit, subject to these constraints.

** For this problem, for each i , $OPT(i)$ is the profit obtained by the best possible selection of locations from those numbered 1 through i . The decision to be made is whether or not to include location i . If we do not, we just need to look at locations 1 through $(i - 1)$. If we do, then we get profit p_i and then we need to look at locations 1 through j , where j is the largest value with $m_j \leq m_i - k$.

(a) Write the recurrence relation for the optimum value for each subproblem.

$OPT(i) = \max(OPT(i - 1), p_i + OPT(j))$; $OPT(0) = 0$ where j is defined as above

(b) Write the pseudocode for your algorithm.

Note: The best way to do this is to pre-calculate all of the j values; this can be done in linear time. Doing so results in a linear algorithm. If this is not done, then at each step, we need to search backwards to find m_j (and thus j) – this leads to a loglinear time algorithm if binary search is used, and a quadratic one if linear search is used.

(c) Explain how to recover the actual solution (the set of locations to build at) from the array of optimal subproblem values.

Starting from the end, compare $OPT(i)$ to $OPT(i - 1)$; if these are the same, we didn't use i , and we should continue with location $i - 1$. If there are not the same, we did use i and we should continue with location j .

(2) In a graph, an *independent set* $S \subseteq V$ is a set of vertices that have no direct edges among them. i.e. $\forall u, v \in S, (u, v) \notin E$. The problem of finding the largest independent set in a graph is an important one; alas, it is NP-complete. However, when the graph is a tree (i.e. connected with no cycles), then the largest independent set can be found in linear time using dynamic programming. (*Hint*: Initially select an arbitrary node r to be the root.)

(a) **In a similar fashion to the statement ** in problem 1**, explain in English the recursive substructure of the problem.

Let r be any node of the tree; consider this as the root and order the rest of the tree appropriately. The subproblem for node v asks for the minimum number of vertices needed to cover the edges in the subtree rooted at v . There are two possibilities: (a) Use node v , in which case we then also need to look at the children of v (since we might need to use two consecutive nodes) or (b) Don't use node v , in which case we must include all of v 's children, and then we inspect the grandchildren of v .

(b) Write the recurrence relation for the optimum value for each subproblem.

$$OPT(v) = \min \left(\begin{array}{l} 1 + \sum_c OPT(c) \text{ where } c \text{ are the children of } v \\ k + \sum_g OPT(g) \text{ where } k \text{ is the number of children of } v \text{ and } g \text{ are the grandchildren of } v \end{array} \right)$$

$OPT(u) = 0$ for any leaf node

(c) The DP array will have an entry for every vertex. In what order should we fill this array?

Start at the leaves, end at the root. (We can obtain this ordering via BFS/DFS.)

(d) Write the pseudocode for your algorithm. Show that it runs in $O(|V| + |E|)$ time. (Note that this is $O(|V|)$ time, since $|E| = |V| - 1$ in every tree.

```
def treeIndSet(T):
    Select arbitrary vertex  $r$  to be the root
    Call BFS/DFS to produce an ordering with leaves first and  $r$  last
    foreach  $v$  in  $T$  in the order produced above:
        if  $v$  is a leaf:
            OPT[v] = 0    # OPT is something like a Python dictionary here
        else:
            A = 1; B = 0
            for each child  $c$  of  $v$ :
                A = A + OPT[c]
                B = B + 1
                for each child  $g$  of  $c$ : # grandchildren of  $v$ 
                    B = B + OPT[g]
            OPT[v] = min(A,B)
    return OPT[r]
```

The BFS/DFS runs in $O(V + E)$ time. The total number of times we execute the $B = B + \text{OPT}[g]$ line in the innermost loop is at most once per node, since each node has at most one grandparent. Similarly, all of the other lines in the program happen at most once per node. Thus the entire large loop takes $O(V)$ time.

(3) Let $x = x_1x_2 \dots x_m$ and $y = y_1y_2 \dots y_n$ be a pair of strings of lengths m and n , respectively. The task is to find the longest common substring – that is, the largest value of k such that indices i and j exist where the strings $x_ix_{i+1} \dots x_{i+k-1}$ and $y_jy_{j+1} \dots y_{j+k-1}$ are identical. (Example: $x = \text{pernicious}$ and $y = \text{supernatural}$. Then $k = 4, i = 1, j = 3$.)

Write an $O(mn)$ dynamic programming algorithm to solve this problem.

(a) **In a similar fashion to the statement ** in problem 1**, explain in English the recursive substructure of the problem.

This is very similar to the edit distance problem from class, but even simpler. The subproblem that we are solving is the longest common substring ending specifically at x_i and y_j (so not the longest one contained in the string to that point). If the letters don't match, that length is zero; otherwise, it is 1 plus the LCS ending with the previous letter in each string. (Note that we will have to remember the best value that we've seen so far as we go.)

(b) Write the recurrence relation giving the value of k for each subproblem.

$$OPT(i,j) = \begin{cases} 0 & \text{if } x_i \neq y_j \\ 1 + OPT(i-1, j-1) & \text{if } x_i = y_j \end{cases} \quad OPT(i,0) = OPT(0,j) = 0$$

(c) Write the pseudocode for this algorithm.

```
def LCS(x,y,m,n):    # assume here strings are 1-indexed
    Declare OPT to be an (m+1) by (n+1) array
    Initialize OPT[i][0] = 0 and OPT[0][j] = 0 for all valid i,j
    best = (0,0,0)    # using a tuple to track value, row, column
    for i = 1 to m:
        for j = 1 to n:
            if x[i] == y[j]:
                OPT[i][j] = 1 + OPT[i-1][j-1]
                if OPT[i][j] > best[0]:
                    best = (OPT[i][j],i,j)
            else:
                OPT[i][j] = 0
    return best
```

(d) Create the actual DP array for the given example (pernicious and supernatural). Be sure to indicate the order in which you filled in the table.

Pretty boring table, in the end:

		S	U	P	E	R	N	A	T	U	R	A	L
P	0	0	0	0	0	0	0	0	0	0	0	0	0
E	0	0	0	1	0	0	0	0	0	0	0	0	0
R	0	0	0	0	2	0	0	0	0	0	0	0	0
N	0	0	0	0	0	3	0	0	0	0	1	0	0
I	0	0	0	0	0	0	4	0	0	0	0	0	0
C	0	0	0	0	0	0	0	0	0	0	0	0	0
I	0	0	0	0	0	0	0	0	0	0	0	0	0
O	0	0	0	0	0	0	0	0	0	0	0	0	0
U	0	0	1	0	0	0	0	0	0	1	0	0	0
S	0	1	0	0	0	0	0	0	0	0	0	0	0

(e) Explain how to recover the indices i and j from the completed array.

Find the largest entry in the table; its indices i and j are the end of the LCS; the start will be at $i - (k - 1), j - (k - 1)$, where k is the value of that largest entry.

(4) Recall from Exam 1 the problem of finding within a list of n arbitrary integers the subsequence of consecutive numbers that has the maximum sum. For example, in the array $[-3 \ 5 \ -8 \ -1 \ \underline{3 \ 7 \ 1 \ 0 \ 6 \ -4 \ 5}] \ -9 \ 0 \ -2 \ 4 \ 1 \ -2]$, the best such subarray is the underlined section, with a total sum of 18.

The naïve brute force algorithm has $O(n^3)$ runtime. This can be improved by using prefix sums to obtain an $O(n^2)$ algorithm, and we also saw that divide-and-conquer can give us an algorithm with $O(n \log n)$ runtime.

However, using dynamic programming, we can solve this problem in linear time. There is one wrinkle: The DP array will not contain the optimal answers to the subproblems of size i . Rather, it should contain the sum of the best possible consecutive subsequence **ending at position i** .

(a) **In a similar fashion to the statement ** in problem 1**, explain in English the recursive substructure of the problem.

This one works similarly to problem 3. The entries in the table are not the best sums anywhere in the sequence up to i – they are the best sums ending specifically at location i . There are two possibilities: (a) if the previous value is negative, just use the value and the current location, or (b) if the previous value is positive, add it to the current value.

(b) Write the recurrence relation for the values stored in the DP array.

$$OPT[i] = \max(A[i], A[i] + OPT[i - 1]); OPT[0] = 0$$

Note: Not required, but here's the actual array for the given example:

current	--	-3	5	-8	-1	3	7	1	0	6	4	5	-9	0	-2	4	1	2
OPT	0	-3	5	-3	-1	3	10	11	11	17	21	26	17	17	15	19	20	22



(c) Write the pseudocode to fill the array correctly.

```
def maxSubseqSum(A,n): # assume A goes from A[1] to A[n]
    Declare OPT to be an array of length n+1
    OPT[0] = 0
    best = (0,0) # again, a tuple for value/location
    for i = 1 to n:
        OPT[i] = A[i]
        if OPT[i-1] > 0:
            OPT[i] += OPT[i-1]
        if OPT[i] > best[0]:
            best = (OPT[i],i)
    return best
```

(d) Once the array is full, how do we find the actual answer? (Both the value of the sum and then the best possible consecutive subsequence.)

Just find (or keep track of) the largest entry in OPT, which is the best sum. It will be at the location of the last value in the best sum. Then walk backwards down the array until you hit a negative number; the location after that negative is the first one in the optimal sum. (See boldface, italics, and array in the array for the example above.)

(e) Show that the full algorithm (including returning the actual solution) can be done in $O(n)$ time.

There's one loop; it executes $O(n)$ times. Everything inside of it is constant time. Finding the final solution involves walking down the OPT array; worst case, it examines n entries.